

For this assignment I used Codex, mainly to talk through class structuring and replayability. It also helped a lot with edge cases and testing. I treated AI's response as suggestions and made my own changes, as its proposed code wasn't always super efficient.

Important AI Interactions:

1. Planning class structure:
 - I consulted the AI on how to separate game infrastructure from specific Mastermind logic in the program. The AI provided some potential classes and interfaces for this purpose. This conversation allowed me to determine the key roles within the program, namely launching, configuration storage, game control, Mastermind feedback computation, and guess results storage.
2. Reviewing the feedback algorithm:
 - I utilized AI in examining how black and white pegs are supposed to be counted when there are repeated colors. The critical consideration here is ensuring that one peg in the secret or in the guess cannot account for more than one feedback. This process involves two steps: the first step deals with exact matches while labeling the pegs as used, while the second step involves checking the unmatched pegs for matching color but wrong position. This was confirmed through duplication of colors, where I guessed RRRR against a secret code of RBGY.
3. Checking test cases
 - I instructed the AI to come up with more boundary tests that are significant in relation to the criteria in the assignment. The software was compiled and tested using the terminal. I used deterministic testing mode, where the secret code was known to ensure predictable console output, instead of randomly generated code.

AI suggestions I changed:

- An early AI-generated design recommended introducing separate ConsoleIO and SecretCodeGenerator classes and additional interfaces for the game and rule behavior. While it would make the components easier to replace in a larger application, I decided against implementing the full suggestion due to the assignment warning to not add abstractions for the sole reason of increasing the number of classes. Given that this version only has one console and one secret-generation method, these extra classes would only add complexity without providing any meaningful code reuse. As such, I left console flow and secret generation inside MastermindGame, separated MastermindRules since validation and feedback mechanisms have a clear responsibility that can be tested in isolation, and kept GameConfiguration separate as the assignment requires its settings to be changeable. The AI further suggested using a frequency map to calculate white pegs, but I opted for a two pass algorithm and boolean arrays since it better follows the Mastermind rules and is easier to trace if there are duplicate colors. Finally, while the AI suggested using JUnit for automated testing, I used a simple MastermindRulesTest class without any external libraries to allow compiling and directly testing the code with javac. These changes allowed me to reduce the design surface

while still retaining flexibility for adding additional abstractions if Part II actually requires them.